

Algorithm Part-04

Polynomial time

Polynomial time refers to the amount of time an algorithm takes to solve a problem, where the time increases at most as a polynomial function of the input size. In simpler terms, if the time taken to solve a problem grows reasonably (not too fast) as the input gets larger, it's considered to be solvable in polynomial time.

Example:

- If you have an algorithm that takes n^2 steps for an input of size n , then for an input of size 10, it will take $10^2=100$ steps. This is polynomial time because the time taken grows relatively slowly as n increases.

Deterministic Algorithms

A deterministic algorithm is one that, given a particular input, will always produce the same output and follow the same sequence of steps.

Characteristics:

- **Predictable:** The behavior and output are predictable.
- **Repeatable:** Running the algorithm multiple times with the same input will yield the same result.
- **Examples:**
 - Sorting algorithms (e.g., QuickSort, MergeSort)
 - Search algorithms (e.g., Binary Search)
 - Mathematical calculations (e.g., finding the greatest common divisor)

Non-Deterministic Algorithms

A non-deterministic algorithm may produce different outputs for the same input on different executions. This can be due to randomization or inherent uncertainty in the process.

Characteristics:

- **Unpredictable:** The behavior and output can vary with each execution.
- **May involve randomness:** Often uses random choices or heuristics.
- **Examples:**
 - Randomized algorithms (e.g., QuickSelect)
 - Algorithms involving heuristics (e.g., genetic algorithms)
 - Non-deterministic Turing machines (theoretical models in computation)

P (Polynomial Time) Problem

- **What it is:** P is the class of problems that can be solved quickly (in polynomial time) using a straightforward method.
- **Example:** Sorting a list of numbers or finding the shortest path in a graph.
- **Key Point:** If a problem is in P, you can find the solution efficiently.

NP (Nondeterministic Polynomial Time) Problem

- **What it is:** NP is the class of problems where, if we have a solution, we can check if it's correct quickly (in polynomial time).
- **Example:** The Traveling Salesman Problem, where you can easily check if a given route is short enough, but finding the best route is hard.
- **Key Point:** If a problem is in NP, it means you can verify a solution quickly, even if finding that solution is tough

NP-hard problem

An **NP-hard problem** is a problem for which:

1. Every problem in NP can be reduced to it in polynomial time.
2. It may not necessarily belong to the class NP (it might not have a verifiable solution in polynomial time).

In summary, NP-hard problems are at least as hard as the hardest problems in NP, and solving one efficiently would imply efficient solutions for all NP problems.

NP-complete problem

An **NP-complete problem** is a decision problem that satisfies two criteria:

1. **Verification:** Given a proposed solution, it can be verified in polynomial time.
2. **Polynomial-Time Reduction:** Every problem in NP can be transformed into this problem in polynomial time.

If any NP-complete problem can be solved in polynomial time, then all problems in NP can also be solved in polynomial time.

Polynomial algorithm

A **polynomial algorithm** is an algorithm whose time complexity can be expressed as a polynomial function of the size of the input. This means that the run time of the algorithm grows at a rate proportional to a polynomial expression in the size of the input data.

Key Characteristics

- **Time Complexity:** The time complexity can be expressed as $O(n^k)$, where:
 - n is the size of the input.
 - k is a constant that represents the degree of the polynomial.
- **Efficiency:** Polynomial algorithms are generally considered efficient, especially when compared to exponential algorithms, which can take significantly longer to run as the input size increases.

Examples

1. Sorting Algorithms:

- **Merge Sort:** $O(n \log n)$
- **Quick Sort** (average case): $O(n \log n)$

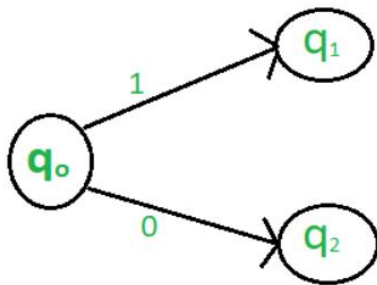
2. Graph Algorithms:

- **Dijkstra's Algorithm** for shortest paths: $O(V^2)$ (where V is the number of vertices)
- **Floyd-Warshall Algorithm** for all pairs shortest paths: $O(V^3)$

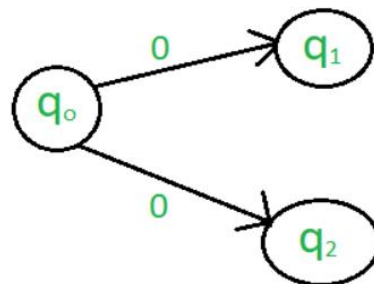
3. Dynamic Programming:

- **Fibonacci Sequence Calculation** using dynamic programming: $O(n)$.

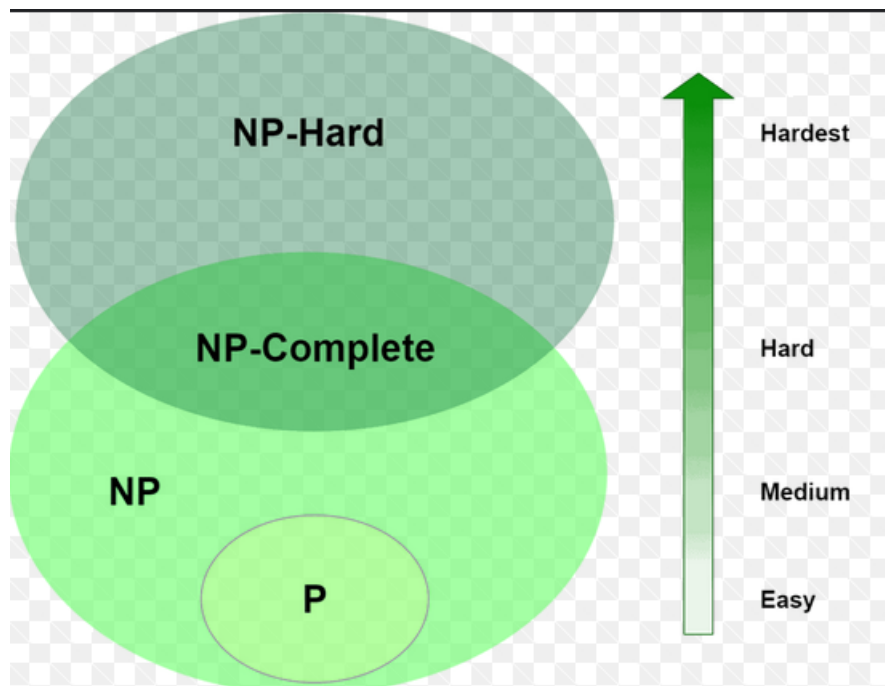
GeekforGeeks



Deterministic Algorithm



Non-Deterministic Algorithm



What are the key differences between deterministic algorithms, non-deterministic algorithms, and polynomial algorithms?

Feature	Deterministic Algorithms	Non-Deterministic Algorithms	Polynomial Algorithms
Output Consistency	Same output for the same input.	May be different outputs for the same input.	Consistent output; runs in polynomial time.
Execution Path	Fixed sequence of steps.	Multiple possible paths.	Fixed sequence; polynomial path.
Predictability	Predictable results.	Unpredictable results.	Predictable results.
Complexity	Easier to analyze; typically efficient.	More complex to analyze.	Efficient; time complexity $O(n^k)$.
Use Cases	Consistency and reliability.	Exploration of multiple solutions.	Efficient problem solving.

What is an NP-hard problem in computational complexity theory?

An **NP-hard problem** is a classification in computational complexity theory that describes problems for which:

- **Difficulty:** They are at least as hard as the hardest problems in NP (nondeterministic polynomial time).
- **Verification:** There is no guarantee that a solution can be checked quickly (in polynomial time).
- **Reduction:** Any problem in NP can be transformed into an NP-hard problem in polynomial time.

In simpler terms, NP-hard problems are those for which finding a solution is challenging, and even verifying a proposed solution may not be efficient. They play a crucial role in understanding computational limits.

When we called a Problem NP-Hard?

A problem is classified as **NP-hard** when it satisfies the following criteria:

1. **At least as hard as the hardest problems in NP:** An NP-hard problem is at least as difficult to solve as the hardest problems in the NP (nondeterministic polynomial time) class. This means that if you could find a polynomial-time solution for an NP-hard problem, you could also solve all NP problems in polynomial time.
2. **No requirement for a solution to be verifiable in polynomial time:** Unlike NP problems, where potential solutions can be verified quickly (in polynomial time), NP-hard problems do not have this requirement. This means that there may not be a feasible way to check the correctness of a solution quickly.
3. **Reduction:** A problem is NP-hard if every problem in NP can be reduced to it in polynomial time. This means that if you can solve the NP-hard problem efficiently, you can also solve all NP problems efficiently.

Why Can't We Solve NP-Hard Problems polynomial time?

We can't solve NP-hard problems in polynomial time for a few key reasons:

1. **Complexity:** NP-hard problems are some of the hardest problems to solve. If we could solve one in polynomial time, it would mean that all problems in NP could also be solved quickly, which is a huge question in computer science (P vs. NP).
2. **Too Many Options:** Many NP-hard problems involve lots of possible solutions, which can grow very quickly as the problem size increases. Checking all these options takes too long.
3. **No Known Fast Methods:** Despite a lot of research, there are no known methods that can solve all NP-hard problems quickly. Most approaches are either slow or only give approximate answers.
4. **Connections to Other Problems:** NP-hard problems are linked to many other difficult problems. Solving one would make it easier to solve many others, which adds to their complexity.

In short, NP-hard problems are very challenging, and finding quick solutions is still an open question in computer science.

Common NP-Hard Problems

Traveling Salesman Problem (TSP)

The Traveling Salesman Problem (TSP) is proven to be NP-hard through a reduction from the Hamiltonian Cycle problem, which is known to be NP-hard. Here's a concise outline:

Proof Outline

1. **Definition of TSP:** Given cities and distances, find the shortest route that visits each city exactly once.
2. **Hamiltonian Cycle Problem:** Determine if a graph has a cycle visiting every vertex exactly once.

3. Reduction:

1.Instance of Hamiltonian Cycle: Let $G=(V,E)$ be an undirected graph with vertices V .

2. Constructing an Instance of TSP:

- Create a complete graph K where each vertex corresponds to a vertex in G .
- Assign edge weights:
 - If there is an edge between two vertices in G , set the distance to 1.
 - If there is no edge, set the distance to a large value (greater than the number of vertices, e.g., $|V|+1$).

3. Relationship:

- If G has a Hamiltonian cycle, there is a tour in the TSP instance of length $|V|$.
- If G does not have a Hamiltonian cycle, any tour will have a length greater than $|V|$.

4. Conclusion

- Since solving TSP for the constructed instance allows us to solve the Hamiltonian Cycle problem, and Hamiltonian Cycle is NP-hard, it follows that TSP is also NP-hard.

Summary

The Traveling Salesman Problem is NP-hard because it can be reduced from the Hamiltonian Cycle problem, demonstrating that no polynomial-time algorithm is known for the general case of the problem.

0/1 Knapsack Problem

The 0/1 Knapsack Problem is a well-known NP-hard problem in combinatorial optimization. Here's a concise proof of its NP-hardness using a reduction from the Subset Sum problem.

Proof Outline

1. Definition of 0/1 Knapsack Problem

- **Problem Statement:** Given a set of items, each with a weight and a value, determine the maximum value that can be obtained by selecting items without exceeding a given weight capacity W . Each item can either be included (1) or excluded (0).

2. Subset Sum Problem

- **Problem Statement:** Given a set of integers and a target integer S , determine if any subset of the integers sums exactly to S .

3. Reduction from Subset Sum to 0/1 Knapsack

1. **Instance of Subset Sum:** Let $\{a_1, a_2, \dots, a_n\}$ be a set of integers and S be the target sum.
2. **Constructing an Instance of 0/1 Knapsack:**
 - Create n items, where each item i has:
 - Weight $w_i = a_i$.
 - Value $v_i = a_i$
 - Set the knapsack capacity $W = S$.
3. **Relationship:**
 - If there is a subset of the integers that sums to S , then selecting the corresponding items will fill the knapsack exactly to its capacity with total value S .
 - Conversely, if the optimal value of the knapsack is S , then the selected items correspond to a subset of integers that sums to S .

4. Conclusion

- Since solving the 0/1 Knapsack Problem allows us to solve the Subset Sum problem, and Subset Sum is NP-hard, it follows that the 0/1 Knapsack Problem is also NP-hard.

Summary

The 0/1 Knapsack Problem is NP-hard because it can be reduced from the Subset Sum problem, demonstrating that no polynomial-time algorithm is known for the general case of the problem.

Coin Toss Problem

1. Problem Formulation:

- Given n coin tosses, can we achieve a specific sequence of heads (H) and tails (T)?

2. Verification in NP:

- Given a sequence of outcomes, counting heads and tails verifies the result in linear time, which is polynomial.

3. Reduction:

- Consider the Subset Sum problem: Given a set of integers, can we find a subset that sums to a specific target? You could construct a corresponding coin toss scenario where each integer corresponds to a specific outcome (H or T) and each target sum corresponds to a required number of heads or tails.

4. Conclusion

If you successfully show the above steps, you would conclude that the coin toss problem is NP-hard. This method hinges on the ability to define the problem clearly and construct a valid reduction from a known NP-hard problem

Cook's Theorem Statement

Cook's Theorem Statement: The satisfiability problem (SAT) is NP-complete. This means that if you can find a polynomial-time algorithm to solve SAT, you can solve all problems in NP in polynomial time.

Implications:

1. If SAT can be solved in polynomial time, then all NP problems can also be solved in polynomial time, implying $P=NP$.
2. If no polynomial-time algorithm exists for SAT, it supports the widely held belief that $P \neq NP$.